CRM Database Design

1. Table (customerss) :

Column	datatype	Constraints
customer_id	INT	PK, auto increement
full_name	VARCHAR(50)	not null
email	VARCHAR(150)	unique
phone	VARCHAR(20)	unique
source_channel	VARCHAR(50)	index (whatsapp, fb, insta,)
created_at	Datetime	default CURRENT TIMESTAMP
updated_at	DATETIME	On Update

* A customer is the unique real-world person - even if they contacted you from multiple channels.

2. Table (lead) :

Column	dtype	Constraints
lead_id	INT	Pk, autoincrement
customer_id	INT	Fk, customerss.customer_id
lead_source	VARCHAR(50)	(whatsapp/fb/insta/linkedin website)
lead_score	~~VARCHAR(50)~~ INT	AI-generated (0-100)
lead_status	VARCHAR(50)	new, contacted, qualified, converted, lost
lead_category	~~VARCHAR~~(50)	Hot, Warm, cold (AI classify)
assigned_to	~~DATETIME~~ INT	Fk, user/agent table (opt)
created_at	DATE TIME	default CURRENT TIMESTAMP
update at	DATETIME	On update

A lead represents one specific inquiry/opportunity tied to a customer. One person can generate multiples leads over time

3. Table Conversation

Column	dtype	Constraints
conversation_id	int	Pk, auto-increment
lead_id	int	Fk, leads.lead-id
Channel	VARCHAR(50)	website chat, whatsapp, FB, insta DM, linkedin
status	VARCHAR(50)	Active, close, Escalated
started_at	DATETIME	Default current_timestamp
ended_at	DATETIME	Nullable

A conversation is a session/thread between AI agent and the lead. One lead can have many conversation (they may reach out again weeks later.

4. Table : messages :

Column	dtype	Constraints
message_id	INT	PK, autoincrement
conversation_id	INT	FK, conversation.conversation_id
sender_type	VARCHAR(20)	Customer / AI agent / human agent
message_text	TEXT	NOT NULL
message_type	VARCHAR(20)	Text, image, file, audio
sent_at	DATETIME	Default Current_timestamp

A message is single line of communication inside a conversation.

Relationship Diagram

Customer ⟶ leads (Many)
↓
Conversation (Many)
↓
Messages.

Customer
↳ has many → lead
↳ has many → conversation
↳ has many → messages.

Lead
↳ has many → lead scoring log (optional)

This is the one-to-many relationship.

AI Customer Support knowledge System

1. Required tables.

- Customers
- products
- Orders
- Order_items
- support_tickets
- conversations
- messages
- faqs

2. Table Structures :

i) customers : stores customer information

Column	Type	key
customer_id	UUID	P/k
name	VARCHAR	
email	VARCHAR	Unique
phone	VARCHAR	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

- customer_id is the primary key and customer table is the central table because almost everything

Products :

Stores the company's products.

column	Type	Key
Product_id	UUID	PK
name	VARCHAR	
description	TEXT	
price	DECIMAL	
stock_quantity	INTEGER	
created_at	TIMESTAMP	

primary key is product_id
this table relatively independent.

Orders : stores customers orders

Column	Type	key
order_id	UUID	PK
customer_id	UUID	FK
order_date	Timestamp ~~UUID~~	
status	VARCHAR	
total_amount	DECIMAL	
shipping_address	TEXT	
created_at	TIMESTAMP	

Customer 1 ——→ N orders
One customer can have many orders.
Fk : orders.customer_id → customers.customer_id.

4. Order items

Column	Type	Key
order_item_id	UUID	PK
order_id	UUID	FK
Product_id	UUID	FK
quantity	integer	
unit_price	Decimal	

Relationship: Order 1 — N order_items
Product 1 — N order_items

This table resolves many-to-many relationship between orders and products

eg Order #1001
↳ laptop x 1
↳ Mouse x 2
↳ keyboard x 1

5. Conversation : stores individual customer-chat sessions

Column	Type	key
conversation_id	UUID	PK
customer_id	UUID	FK
started_at	TIMESTAMP	
ended_at	TIMESTAMP	
status	VARCHAR	
channel	VARCHAR	

Relationship : If customer can have many conversation.

6. Support Tickets : stores customer problems/ issues

Column	Type	key.
ticket_id	UUID	Pk
conversation_id	UUID	Fk
customer_id	UUID	Fk
subject	VARCHAR	
issue_description	TEXT	
priority	VARCHAR	
status	VARCHAR	
created_at	TIMESTAMP	
resolved_at	TIMESTAMP	

Relationships :

customer 1 — N support tickets
conversation 1 → N support tickets.

7. Messages : Stores individual messages inside conversations

Column	Type	Key
message_id	UUID	Pk
conversation_id	UUID	Fk
sender_type	VARCHAR	
message_text	TEXT	
created_at	TIMESTAMP	

sender_type could be customer, ai_agent, human_agent.

Relationship : Conversation 1 — N messages.

8. FAQs :

Column	Type	Key
faq_id	UUID	PK
question	TEXT	
answer	TEXT	
category	VARCHAR	
is_active	BOOLEAN	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

- For the basic design FAQs can remain independent.

One-to-many Relationship

- customer 1 — N order
- Customer 1 — N conversation
- customer 1 — N support tickets
- conversation 1 — N messages
- Order 1 — N order items
- Product 1 — N order items
- conversation 1 — N support tickets.

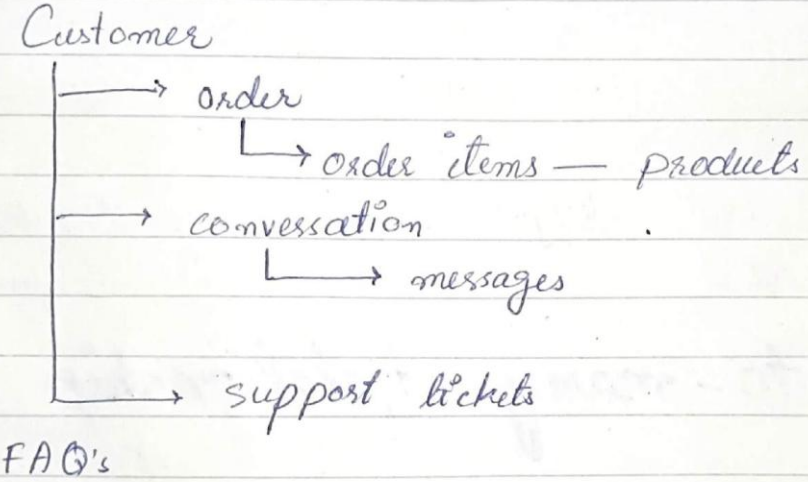
Independent Tables:

In the basic design:
Product can exist before anyone order them.

faqs can exist independently of customers, orders, or conversations.

∴ However, orders, tickets, conversations, messages and order_items are dependent on other ~~entities~~ entities.

Tables Should be connected



How can AI answer the question?

Query: What was my previous order and why was my support ticket opened.

Customer → Find customer_id → Find previous orders → Find order_items → Find products → Find support ticket → Find related conversation → Read messages → give answer.

eg : Database might contain

customer :
Tehmina

Previous order :
Order #1025
Date : 30- August
Product : laptop
status : delivered

Support Ticket :
Ticket #501
subject : Delivery Issue
Reason :
Customer reported that the order arrived later than expected.

AI can generate :

" Your previous order was #1025 for laptop, placed on August 30. Your support ticket was opened because you reported an issue with the delivery "

E-R Diagram